EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Aim

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Programming Language: C
- Debugger / System Tool: Ptrace
- Compiler: GCC
- System Requirements:
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Introduction

Code injection is a technique where malicious code is inserted into a running process to alter its execution. ptrace is a Linux system call that allows one process to observe and control another process, commonly used by debuggers. Understanding ptrace-based code injection helps analyze malware behavior, reverse engineering techniques, and system-level security vulnerabilities. Kali Linux provides a secure environment to study such low-level attacks ethically.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.
2. Create a simple C program:
3. nano target.c
4. Write a basic infinite loop program.

```
#include <stdio.h>

int main() {
   while(1) {
      printf("This is an infinite loop...\n");
   }
   return 0;
}
```

5. Compile the program:
6. gcc target.c -o target

Step 3: Run the Target Application

1. Execute the program:
2. ./target
3. Note the Process ID (PID) using:
4. ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:
2. nano injector.c
3. Write ptrace-based code to attach to the target process and modify its memory or registers.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <unistd.h>

int main(int argc, char *argv[]) {

   if (argc != 2) {
      printf("Usage: %s <pid>\n", argv[0]);
      return 1;
   }

   pid_t pid = atoi(argv[1]);
   struct user_regs_struct regs;

   // Attach to target process
   if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
      perror("ptrace attach failed");
```

```
        return 1;
    }

    // Wait for process to stop
    waitpid(pid, NULL, 0);
    printf("Attached to process %d\n", pid);

    // Get registers
    if (ptrace(PTRACE_GETREGS, pid, NULL, &regs) == -1) {
        perror("getregs failed");
        return 1;
    }

    printf("Original RIP: %llx\n", regs.rip);

    // Modify instruction pointer (example: +2 bytes)
    regs.rip += 2;

    if (ptrace(PTRACE_SETREGS, pid, NULL, &regs) == -1) {
        perror("setregs failed");
        return 1;
    }

    printf("Modified RIP: %llx\n", regs.rip);

    // Detach process
    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    printf("Detached from process\n");

    return 0;
}
```

4. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
2. gcc injector.c -o injector

Step 6: Attach to Target Process

1. Attach injector to target process using PID:
2. sudo ./injector <PID>
3. Injector gains control of the target process.

Step 7: Inject and Execute Code

1. Modify target process memory or registers.
2. Resume execution of the target process.

3. Observe injected behavior.

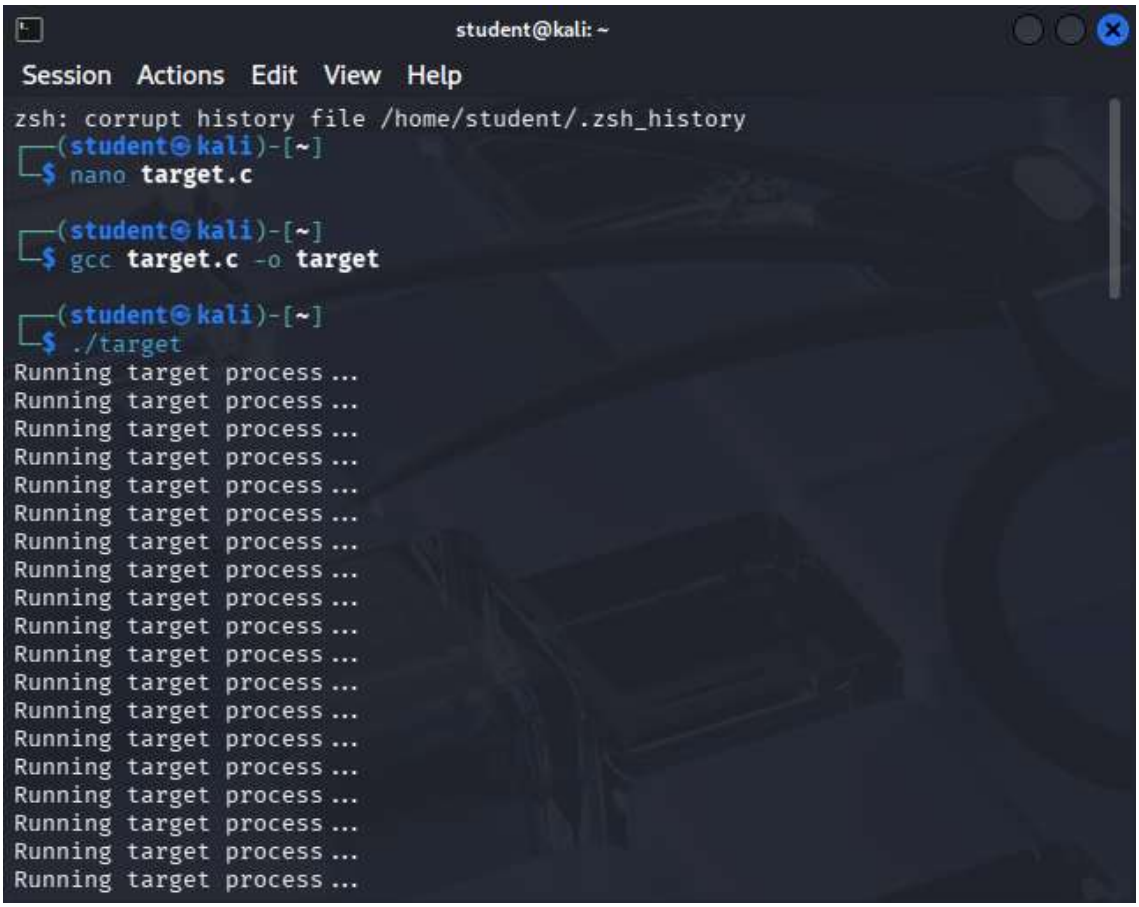
Output

- Injector successfully attaches to the running process.
- Target application behavior is modified during execution.
- Code injection is observed in real time.

```
Session  Actions  Edit  View  Help
  GNU nano 8.6
#include <stdio.h>
#include <stdlib.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <pid>\n", argv[0]);
        return 1;
    }

    pid_t pid = atoi(argv[1]);
    struct user_regs_struct regs;

    // Attach
    if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
        perror("Attach failed");
        return 1;
    }

    waitpid(pid, NULL, 0);
    printf("Attached to PID: %d\n", pid);

    // Get registers
    ptrace(PTRACE_GETREGS, pid, NULL, &regs);
    printf("Original RIP: %llx\n", regs.rip);

    // Modify RIP
    regs.rip += 2;

    ptrace(PTRACE_SETREGS, pid, NULL, &regs);
    printf("Modified RIP: %llx\n", regs.rip);

    // Detach
    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    printf("Detached\n");

    return 0;
}
```

```
student@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.6                    target.c
#include <stdio.h>
#include <unistd.h>

int main() {
    while(1) {
        printf("Running target process ... \n");
        sleep(1);
    }
    return 0;
}

[ Bad lock file is ignored: ./.target.c.swp ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify
```

```
student@kali: ~
Session  Actions  Edit  View  Help
┌──(student㉿kali)-[~]
└─$ ps aux | grep target
student     6928  0.0  0.0   2568  1428 pts/0    S+   16:28   0:00 ./target
student     7508  0.0  0.1   6544  2244 pts/1    S+   16:29   0:00 grep --col
or=auto target

┌──(student㉿kali)-[~]
└─$ nano injector.c

┌──(student㉿kali)-[~]
└─$ gcc injector.c -o inject

┌──(student㉿kali)-[~]
└─$ ./inject 6928
Attached to PID: 6928
Original RIP: 7fb86dd66687
Modified RIP: 7fb86dd66689
Detached

┌──(student㉿kali)-[~]
└─$

┌──(student㉿kali)-[~]
└─$
```



Result

Code injection using the ptrace system call was successfully demonstrated in Kali Linux.

The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.